

1-Serve your application securely with nginx+modsecurity

1. Start a centos8 container
2. Compile nginx with modsecurity and Use all Core Rule Set(crs)
3. Serve nginx with https protocol
4. Configure all of things using ansible playbook.Such as nginx.conf or modsecurity.conf.
5. Use 3 of least [OWASP TOP 10](#) for testing on your application.

Use our [Flask Web Application Project](#) for this.

2-ARP Spoofing Attack



- Introduction
- Objective
- Steps
- Required Tools
- Result
- Keywords

Introduction

The Address Resolution Protocol (ARP) is a communication protocol used for discovering the link layer address, such as a MAC address, given an IP address. The ARP protocol is a very simple protocol, and it does not implement any security measure. The ARP cache poisoning attack is a common attack against the ARP protocol. Under such an attack, attackers can fool the victim into accepting forged IP-to-MAC mappings. This can cause the victim's packets to be redirected to the computer with the forged MAC address.

Objective

The objective of this task is for interns to gain the first-hand experience on the ARP cache poisoning attack, and learn what damages can be caused by such an attack. In particular, interns will use the ARP attack to launch a man-in-the-middle attack, where the attacker can intercept and modify the packets between the two victims A and B.

Steps

The structure may contain 3 devices namely Attacker, Victim and Server. The aim of the attacker is sniffing packets that belong to communication between the two victims. The attack lab provided. You should follow steps given below:

1. Unzip [arp_spoofing.zip](#)
2. Change directory to `arp_spoofing/` and be sure about `vagrantfile` is placed in here.
3. Use command to initiate the lab: `vagrant up`
4. After successful initialization you can reach VMs with `vagrant ssh <name>`
5. Run the client file in Victim with command `python3 client/client.pyc` and the server file in Server with command `python3 server/server.pyc`.
6. Be sure that you should run server before client and after connection is established you should see an output in Server machine.
7. The lab setup is ready now. You can try to attack Victim and Server and to get message packets in communication.

The vagrantfile is also given below:

```

Vagrant.configure("2") do |config| #
  Attacker VM
  config.vm.define "attacker" do |attacker| attacker.vm.box
    = "generic/alpine38"
    attacker.vm.network "private_network", ip: "192.168.33.10" attacker.vm.provider
    "virtualbox" do |vb|
      vb.name = "attacker" end
  end
  # Server VM
  config.vm.define "server" do |server| server.vm.box =
    "generic/alpine38"
    server.vm.network "private_network", ip: "192.168.33.20" server.vm.provider
    "virtualbox" do |vb|
      vb.name = "server" end
  # Custom server directory
  server.vm.synced_folder "server/", "/home/vagrant/server"
  server.vm.provision "shell", inline: <<-SHELL
    # Update package lists apk update
    # Install Python 3.8 and pip apk add
    python3
  SHELL
end
  # Victim VM
  config.vm.define "victim" do |victim| victim.vm.box =
    "generic/alpine38"
    victim.vm.network "private_network", ip: "192.168.33.30" victim.vm.provider
    "virtualbox" do |vb|
      vb.name = "victim" end
  # Custom victim directory
  victim.vm.synced_folder "client/", "/home/vagrant/client"
  victim.vm.provision "shell", inline: <<-SHELL
    # Update package lists apk update
    # Install Python 3.8 and pip apk add
    python3
  SHELL
end end

```

Required Tools

To perform attack you should use Scapy, a python library for networking. You can also use `tcpdump` for monitoring the packet stream. Wireshark can be used for inspect every packet in given network.

Result

In the result, it is expected from you, done a succesfull attack and monitor traffic between Victim and Server as an attacker. Finally, you should report the content of messages with using wireshark to show that how to attack perfrom.

Keywords

arp, arpspoofing, scapy, tcpdump, vagrant, wireshark

3-Automated REST API Testing with Postman

- Objective
- Requirements
- Tasks
 - Set up Postman environment and collection:
 - Develop test scenarios:
 - Write test scripts:
 - Run automated tests:
- Deliverables

todo: Decide API to be tested

Objective

To develop a suite of automated tests for a REST API using Postman, to ensure that the API functions as intended, and to detect any errors or issues before they are deployed to production.

Requirements

- Understanding of REST API principles
- Knowledge of Postman and its features
- Familiarity with test automation concepts and frameworks
- Knowledge of scripting languages such as JavaScript

Tasks

1. Set up Postman environment and collection:

- Create a new environment in Postman to store variables and configuration settings
- Create a new collection to group related requests and tests

2. Develop test scenarios:

- Review the API documentation of  and identify the key functionalities to be tested (Signup for a free Todoist account to use API)
- Define test scenarios that cover various aspects of the API, including positive and negative test cases
- Create requests for each scenario in the Postman collection, using appropriate HTTP methods (GET, POST, PUT, DELETE, etc.)

3. Write test scripts:

- Write test scripts using Postman's built-in scripting language, JavaScript
- Define test assertions that verify the expected response from each API call
- Use variables from the Postman environment to parameterize test data

4. Run automated tests:

- Run the tests in the Postman collection using the "Runner" feature
- View the results of the test runs and identify any failed tests or errors
- Debug failed tests and update test scripts as necessary

Deliverables

- Postman environment and collection
- Test scenarios and scripts
- Test results and reports

4-Create an Iptables Firewall

- Tasks
- Notes
- Topology

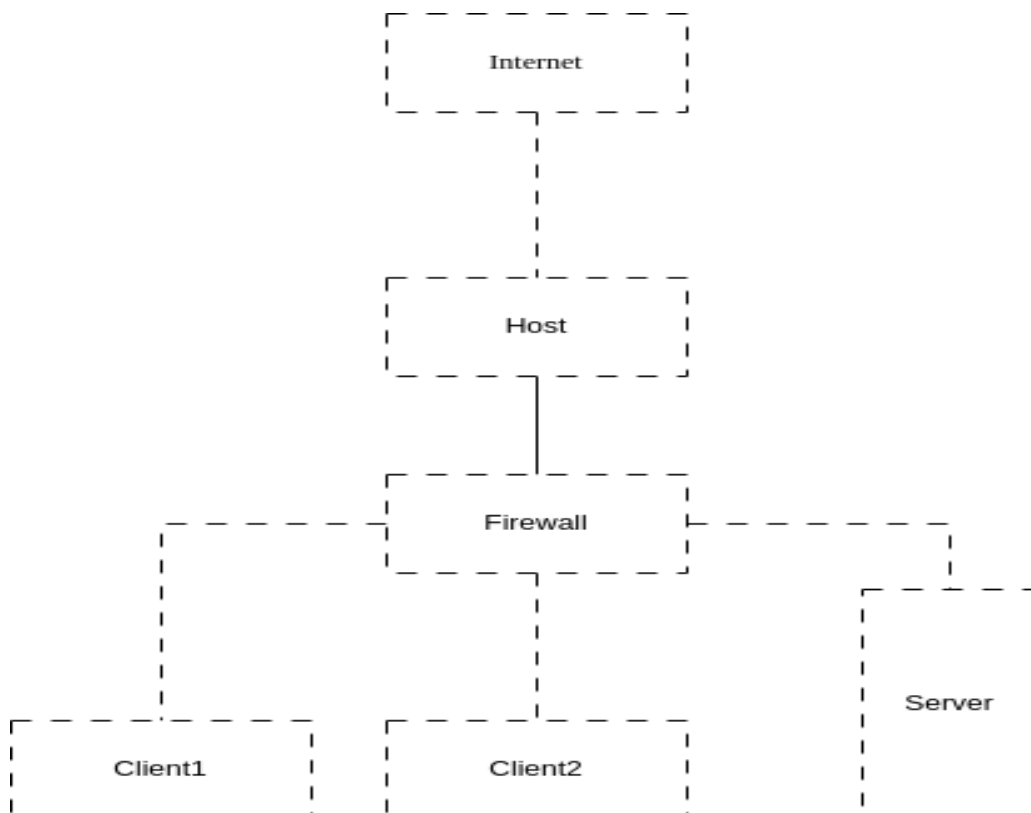
Tasks

1. Create 4 network namespaces.
2. Namespaces are client1, client2, server, firewall
3. Create veth for all namespaces and your host-to-firewall for network communication.
4. Serve sample http service inside the server namespace
5. Create iptables rules inside the firewall namespace and control traffic between the namespaces.
6. Rules
 - a. Client1 can ping to server
 - b. Client2 can access to server for http
 - c. Client2 can ping to firewall
 - d. Client1 doesn't have ping permission to firewall
 - e. Client and server networks are can be access to the internet from firewall namespace via your host machine.

Notes

- Client1 subnetwork is 192.0.2.0/26
- Client2 subnetwork is 192.0.2.64/26
- Server subnetwork is 192.0.2.128/26
- Host-To-Firewall Subnetwork is 192.0.2.192/26
- Firewall should be a stateful.

Topology



5-Find best game

My favorite game hiding inside the pcap file and your target is that. You should use pure python for this mission.

Tips

You can start from this project https://github.com/tintinweb/scapy-ssl_tls and specially this example https://github.com/tintinweb/scapy-ssl_tls/blob/master/examples/sessionctx_sniffer.py

- [best_game.pcap](#)
- [best_game_tls1_3.pcap](#) (you can use wireshark)
- [server.key](#)

6-Flask Web Application Project

- Tasks
- Make it more complicate

Tasks

1. Create a RESTFUL flask application using json data format.
2. Use CRUD for endpoints.
3. Endpoints are

```
/login  
/logout  
/user/list  
/user/create  
/user/delete/{id}  
/user/update/{id}  
/onlineusers
```

4. Data

```
/user/create  
{  
  "username": "",  
  "firstname": "",  
  "middlename": "",  
  "lastname": "",  
  "birthdate": "",  
  "email": "",  
  "password": ""  
}  
/onlineusers  
{  
  "username": "",  
  "ipaddress": "",  
  "logindatetime": ""  
}
```

5. Store data in postgresql.
6. Password should be store salted with sha256.
7. Password complexity should be least [A-Za-z0-9] and min 8 characters.
8. Data should be validates. Example: email format.
9. Log all activities.
10. Serve application using uwsgi and nginx.
11. Use sqlalchemy for db crud implementation

Make it more complicate

Run flask application with more instance using docker. Create active-active cluster database using bucardo.

7-Network routing on distributed container via Open vSwitch

- Tasks
- Topology

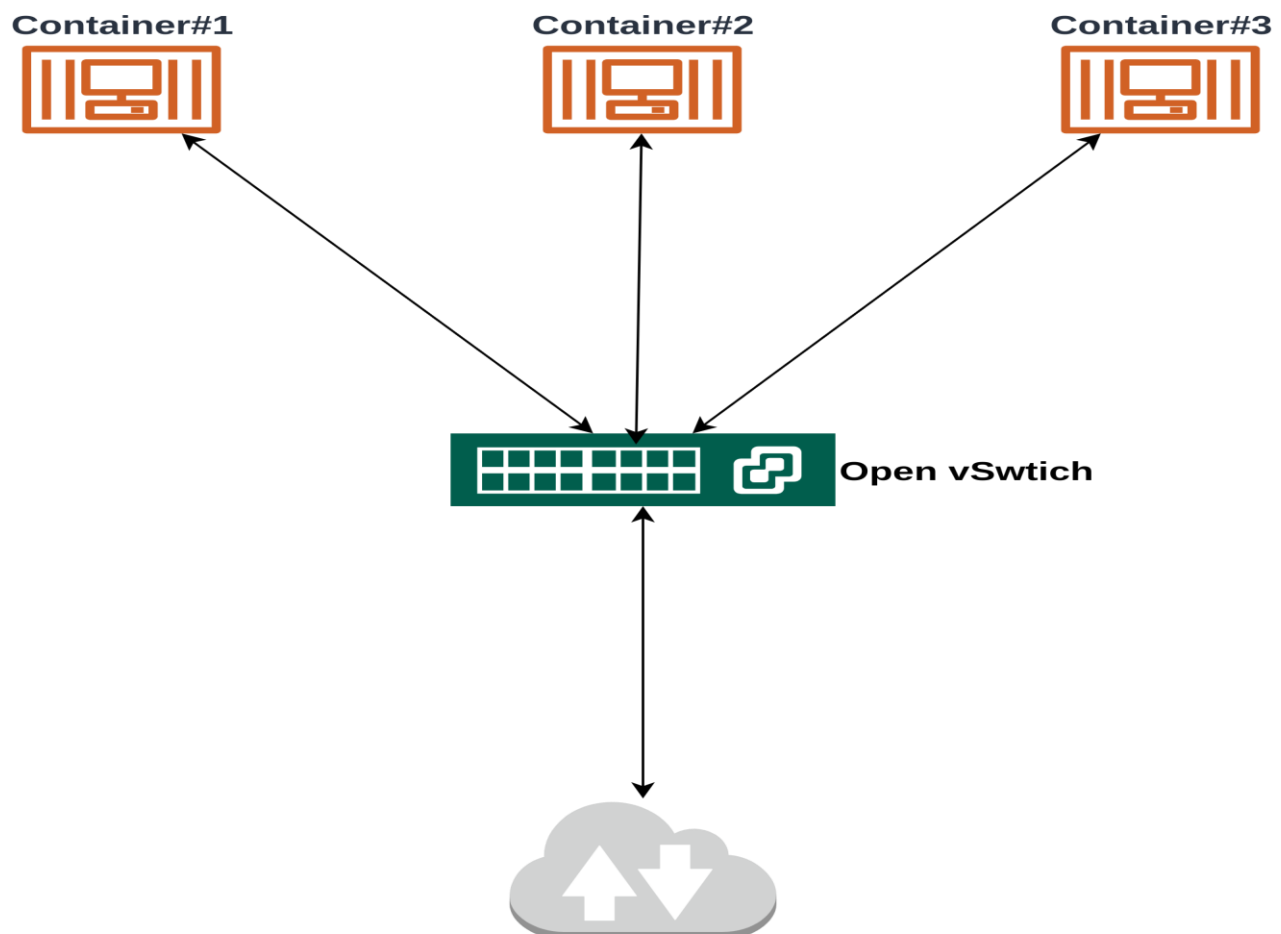
Configuring Three Docker Containers behind OpenvSwitch with HTTP Request Rate Limit and XDP Program

Tasks

Your task includes the following:

1. Configure three Docker containers: You need to set up three Docker containers and ensure they are properly configured to work behind OpenvSwitch.
2. Define HTTP Request Rate Limit: You will need to define HTTP request rate limit for the containers' traffic. This will involve setting up a rate limiting mechanism to control the number of HTTP requests that can be sent to and from each container.
3. Drop Packets with Keyword using XDP Program: You will be required to develop an XDP program to drop packets that contain a specific keyword. The program should be able to monitor traffic to and from the containers and drop packets that match the defined keyword.
4. Testing and Documentation: Finally, you need to perform testing to ensure that the configurations are working correctly. Use Wireshark to monitor dropped packets and verify rate limiting is working as expected. You will also need to document the configurations.

Topology



8-Prepare RPM package

- Doc Links
- Projects
- Expected Result
- Notes

Follow those links and create rpm packages on Centos8 for the projects.

Doc Links

<https://rpm-packaging-guide.github.io/>

<https://docs.fedoraproject.org/en-US/packaging-guidelines/RPMMacros/>

Projects

<https://github.com/synfinatic/netflow2ng>

<https://marlam.de/msmtp/>

Expected Result

All projects will be install via dnf package updater with all dependencies.

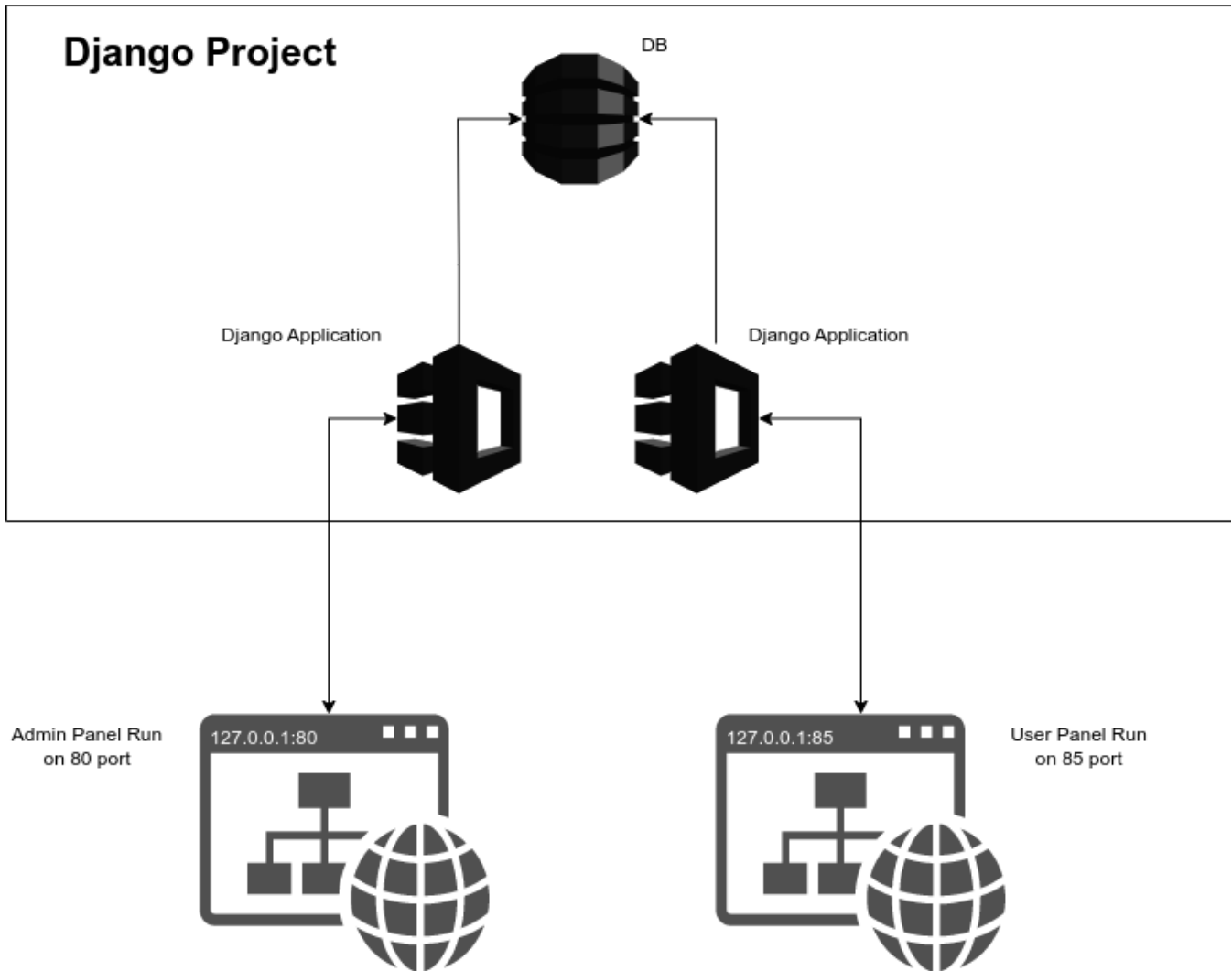
Notes

You can use docker container for Centos8.

9-Run Django on Multiple Ports

- Create a website which suitable with MVC architecture on Django.
- In this website; serve your application from multiple ports which are 80 and 85.
- Think that one of them is admin panel and second one is the user panel.
- You can select any db but highly encouraged to the mysql.
- You should block normal user to the access admin panel.
- You can use apache or nginx web server if you want furthermore.

Topology:



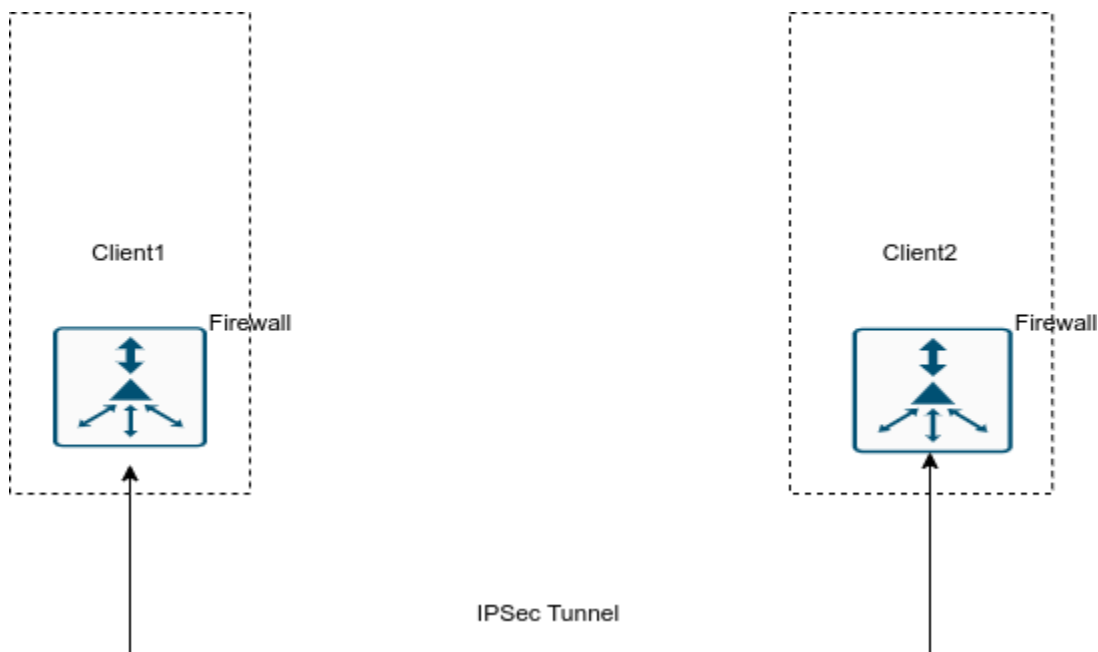
10- Set Up Ipsec-based VPN with Strongswan on CentOS 8

- Tasks
- Topology

Tasks

1. Install 2 CentOS 8 virtual machine via virtualbox (<https://www.centos.org/download/>)
 - a. Client1
 - b. Client2
2. Set up firewalld configurations on each clients.
3. Install & Configure Strongswan on each clients.
4. Configure an IPsec Tunnel via strongswan between those clients.
5. Try to ping one client to another.
6. Watch packet flow via strongswan daemon.

Topology



11- User Test with Selenium IDE and Automation using Selenium

- Introduction
- Objectives
- Methodology
- Tools Used

Introduction

In this project, we will be testing [PHPTRAVELS](#) using Selenium IDE to record and create user tests. We will then automate these tests using Selenium WebDriver.

Objectives

1. To create user tests for the PHPTRAVELS using Selenium IDE ([Check test scenarios document](#)).
2. To automate the user tests using Selenium WebDriver.
3. To ensure the application functions as intended by performing automated tests.

Methodology

1. Setting up the PHPTRAVELS demo account form [link](#).
2. Recording user tests: We will use Selenium IDE to record user tests .
3. Writing test scripts: We will convert the recorded user tests into Python test scripts using Selenium WebDriver. Test will be ran for different browsers (Chrome, Firefox, Edge, IE, Safari)
4. Test automation: We will automate the test scripts using Selenium WebDriver to perform automated testing of PHPTRAVELS.
5. Identifying issues: We will analyze the results of the automated tests to identify any bugs or issues PHPTRAVELS.

Tools Used

1. Selenium IDE: A Firefox/Chrome add-on used to record and create user tests for web applications.
2. Selenium WebDriver: A tool used to automate web browser actions for testing web applications.
3. Python: A programming language used for scripting and test automation.